

# LogGuardian — Rapport de Projet Annuel

---

Plateforme AIOps de détection d'anomalies par Deep Learning

Étudiants : Wassim Torjmen (et collègue ML) Cours : ESGI — 5ème année Année : 2025–2026 Date de remise : 1er mai 2026

## Table des matières

---

1. [Résumé exécutif](#)
2. [Contexte et objectifs](#)
3. [Architecture globale](#)
4. [Phase 1 — Infrastructure cloud \(AWS / EKS\)](#)
5. [Phase 2 — Pipeline de données streaming](#)
6. [Phase 3 — Modèle de Machine Learning](#)
7. [Phase 4 — Interface de monitoring](#)
8. [Déploiement Kubernetes](#)
9. [Sécurité et IAM](#)
10. [Pipeline CI/CD](#)
11. [Résultats et métriques](#)
12. [Difficultés rencontrées et solutions](#)
13. [Améliorations futures](#)
14. [Conclusion](#)
15. [Annexes](#)

## 1. Résumé exécutif

---

**LogGuardian** est une plateforme **AIOps** (Artificial Intelligence for IT Operations) qui détecte automatiquement les anomalies dans les logs système d'une infrastructure distribuée, en temps réel, à l'aide d'un modèle de Deep Learning (LSTM Autoencoder).

### Caractéristiques clés

| Aspect           | Réalisation                                   |
|------------------|-----------------------------------------------|
| Architecture     | Microservices, event-driven, streaming Kafka  |
| Cloud provider   | AWS (EKS, S3, ECR, CodeBuild, IAM)            |
| Orchestration    | Kubernetes (EKS Auto Mode)                    |
| Détection ML     | LSTM Autoencoder PyTorch, seuil percentile 95 |
| CI/CD            | AWS CodeBuild — build + push ECR automatique  |
| Visualisation    | Dashboard temps réel Plotly Dash              |
| Datasets traités | Linux, SSH, Hadoop, BlueGene/L Supercomputer  |
| Volume traité    | 1.4M+ logs en production live                 |

## Chiffres clés (données de production)

- 1 463 500 logs traités par l'ETL
- 105 135 anomalies détectées par règles (ERROR/FATAL)
- 14 893 anomalies détectées par le modèle ML
- 172 anomalies de sévérité haute (ratio > 1.3x du seuil)
- 4 sources de logs en parallèle (Linux, SSH, Hadoop, Supercomputer)
- ~1 000 messages/10s en débit constant

## 2. Contexte et objectifs

### 2.1 Problématique

Les infrastructures modernes génèrent des volumes massifs de logs (terabytes par jour). La détection manuelle d'incidents est :

- **Trop lente** : un humain ne peut pas surveiller des millions de logs
- **Subjective** : un `ERROR` n'est pas toujours un vrai incident
- **Réactive** : on découvre les pannes après les utilisateurs

### 2.2 Solution apportée

Une plateforme automatisée qui :

1. **Ingère** les logs en temps réel depuis multiples sources
2. **Normalise** et enrichit chaque entrée
3. **Détecte** deux types d'anomalies :

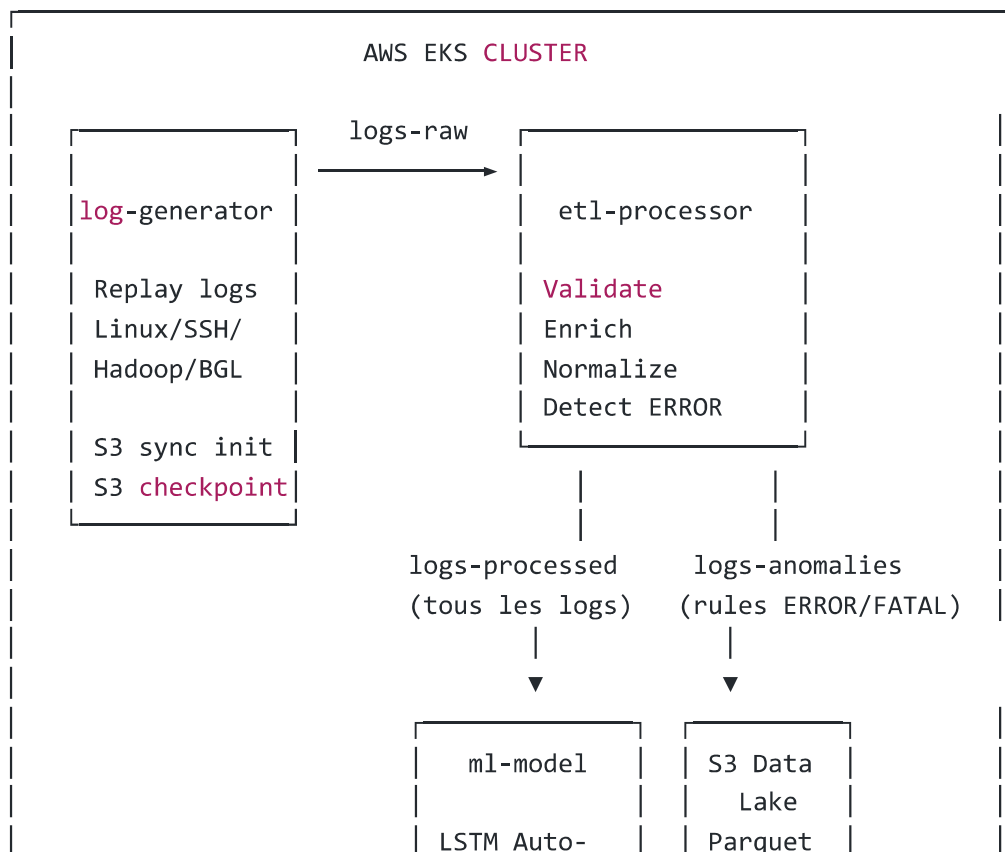
- **Règles** : niveau ERROR / FATAL (rapide, beaucoup de faux positifs)
  - **ML** : patterns sémantiques anormaux (contextuel, plus précis)
4. Visualise les alertes via un dashboard temps réel
  5. Stocke historiquement sur S3 pour analyse rétrospective

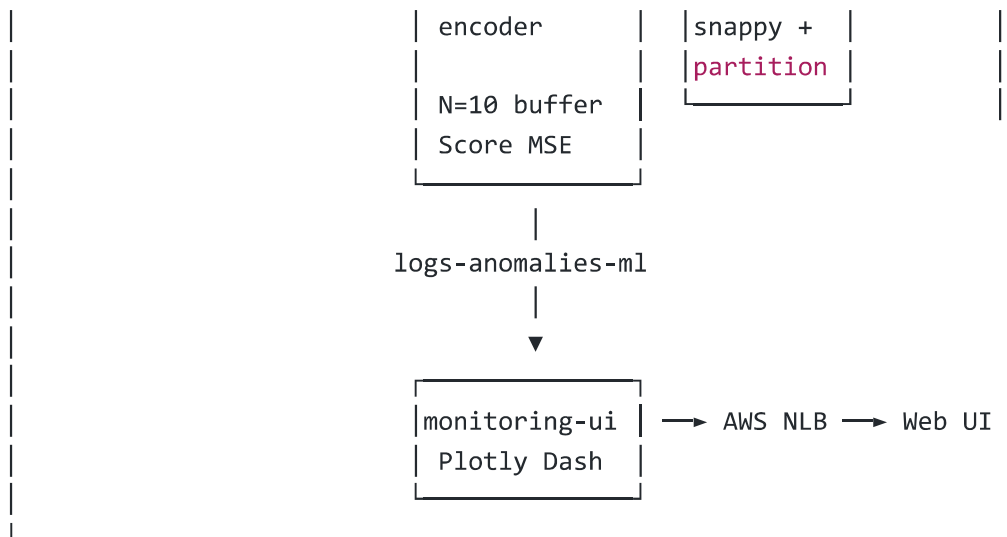
## 2.3 Objectifs techniques

| Objectif             | Métrique cible               | Atteint                   |
|----------------------|------------------------------|---------------------------|
| Latence de détection | < 30 secondes                | ✓ ~5–15s                  |
| Throughput           | > 500 msg/s                  | ✓ 1000 msg/10s soutenu    |
| Disponibilité        | Auto-recovery sur crash      | ✓ Kubernetes restart      |
| Persistance données  | Datalake structuré           | ✓ Parquet partitionné S3  |
| Sécurité             | Pas de credentials hardcodés | ✓ IRSA (IAM Roles for SA) |
| Reproductibilité     | CI/CD automatique            | ✓ CodeBuild sur git push  |

## 3. Architecture globale

### 3.1 Vue d'ensemble





## 3.2 Stack technique

| Couche             | Technologie                     |
|--------------------|---------------------------------|
| Orchestration      | Kubernetes 1.28 (EKS Auto Mode) |
| Cloud              | AWS (eu-west-1)                 |
| Streaming          | Apache Kafka 7.5 + Zookeeper    |
| Stockage           | S3 (Parquet snappy partitionné) |
| Container Registry | Amazon ECR                      |
| CI/CD              | AWS CodeBuild                   |
| ML Framework       | PyTorch 2.x + scikit-learn      |
| Backend services   | Python 3.11                     |
| UI                 | Plotly Dash + confluent-kafka   |
| IaC manuel         | YAML manifests + AWS Console    |

## 3.3 Topics Kafka

| Topic             | Producteur    | Consommateur  | Rôle                  |
|-------------------|---------------|---------------|-----------------------|
| logs-raw          | log-generator | etl-processor | Logs bruts JSON       |
| logs-processed    | etl-processor | ml-model      | Logs enrichis pour ML |
| logs-anomalies    | etl-processor | (futur)       | Détection règles      |
| logs-anomalies-ml | ml-model      | monitoring-ui | Détection ML          |

## 4. Phase 1 — Infrastructure cloud (AWS / EKS)

### 4.1 Ressources AWS provisionnées

| Ressource     | Identifiant                                                          | Rôle                             |
|---------------|----------------------------------------------------------------------|----------------------------------|
| Compte AWS    | 148761640356                                                         | Compte principal                 |
| Région        | eu-west-1 (Irlande)                                                  | Latence Europe                   |
| Cluster EKS   | logguardian                                                          | Orchestration K8s                |
| Node Group    | logguardian-nodes                                                    | EC2 workers                      |
| OIDC Provider | oidc.eks.eu-west-1.amazonaws.com/id/1B3FC4C4EF0C07B7E44948A0E5644D55 | Auth IRSA                        |
| ECR Repos (4) | logguardian/{log-generator,etl-processor,ml-model,monitoring-ui}     | Images Docker                    |
| S3 Datalake   | logguardian-datalake-148761640356                                    | Logs Parquet + raw + checkpoints |
| S3 Models     | logguardian-models-148761640356                                      | Artefacts ML                     |

### 4.2 Configuration EKS

- **Mode** : EKS Auto Mode (gestion automatique des nodes)
- **Networking** : VPC AWS managé, subnets publics + privés
- **Load Balancer** : NLB (Network Load Balancer) internet-facing pour monitoring-ui
- **Storage** : EmptyDir volumes (pour init container sync S3)

### 4.3 Namespace Kubernetes

Tous les services sont déployés dans le namespace `logguardian` pour isolation logique.

## 5. Phase 2 — Pipeline de données streaming

### 5.1 Service `log-generator`

**Rôle** : Simuler un environnement de production en rejouant de vrais fichiers de logs vers Kafka.

#### Fonctionnement

1. **Init container** : `aws s3 sync` pour télécharger les fichiers logs depuis S3 ( `raw-logs/` ) vers un volume `emptyDir` partagé
2. **Main container** : lit les logs via parsers spécialisés et publie sur `logs-raw`
3. **Boucle infinie** : rejoue les logs en continu (cycle `linux` → `ssh` → `hadoop` → `supercomputer`)
4. **Checkpoint S3** : sauvegarde la position toutes les 1000 messages, permet la reprise après redémarrage

## Parsers implémentés

| Parser              | Fichier source                     | Volume       | Format         |
|---------------------|------------------------------------|--------------|----------------|
| LinuxParser         | <code>Linux.log</code>             | ~50K lignes  | syslog         |
| SSHParser           | <code>SSH.log</code>               | ~600K lignes | OpenSSH        |
| HadoopParser        | <code>hadoop/application_*/</code> | ~50K lignes  | Apache Hadoop  |
| SupercomputerParser | <code>supercomputer/BGL.log</code> | ~5M lignes   | IBM BlueGene/L |

## Format de sortie unifié (LogEntry)

```
{
  "timestamp": "2026-05-01T15:37:09",
  "source": "linux",
  "host": "combo",
  "level": "ERROR",
  "component": "kernel",
  "message": "session opened for user root",
  "raw": "Apr 14 10:23:45 combo kernel: session opened..."
}
```

## Innovations techniques

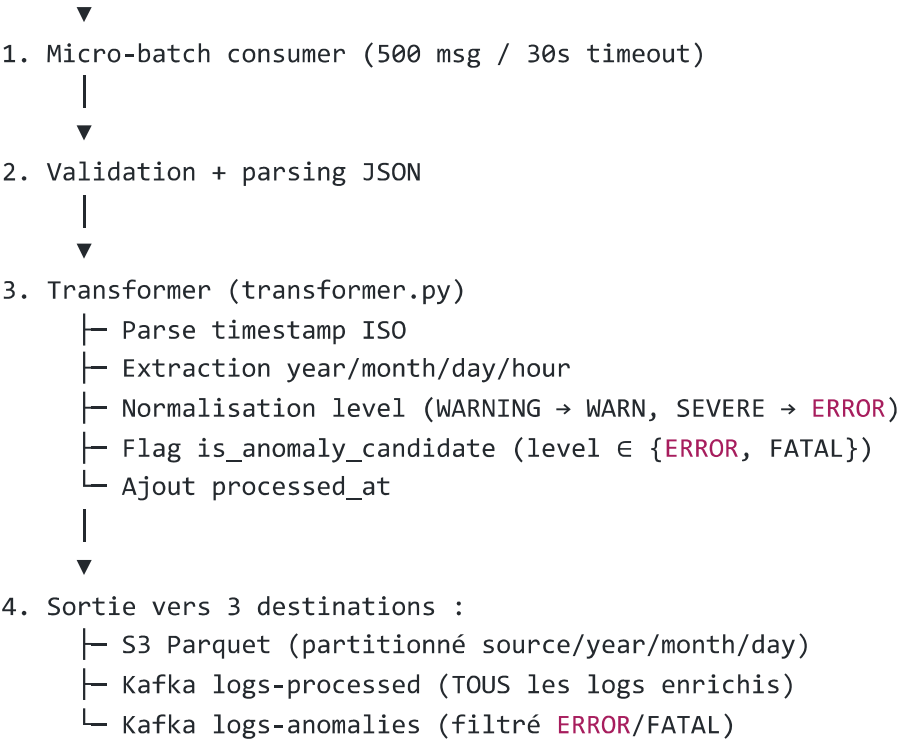
- **Init container pattern** : sépare le download de données du runtime applicatif
- **IRSA** (IAM Roles for Service Accounts) : pas de credentials hardcodés, accès S3 via OIDC
- **Checkpoint S3** : reprise gracieuse après `scale-down` → `scale-up` sans rejouer depuis zéro
- **Signal handling** : `SIGTERM` intercepté pour flush + checkpoint avant arrêt

## 5.2 Service `et1-processor`

Rôle : Consommer les logs bruts, les enrichir et les router vers les bons topics.

### Pipeline de traitement

```
Kafka logs-raw
|
```



Format Parquet S3

```
s3://logguardian-datalake-148761640356/logs/
└─ source=linux/
    └─ year=2026/
        └─ month=05/
            └─ day=01/
                └─ batch_20260501_154410_731652.parquet
```

- **Compression** : Snappy (rapide, bon ratio)
- **Engine** : pyarrow
- **Partitionnement Hive-compatible** : compatible Athena/Glue/Presto

Configuration

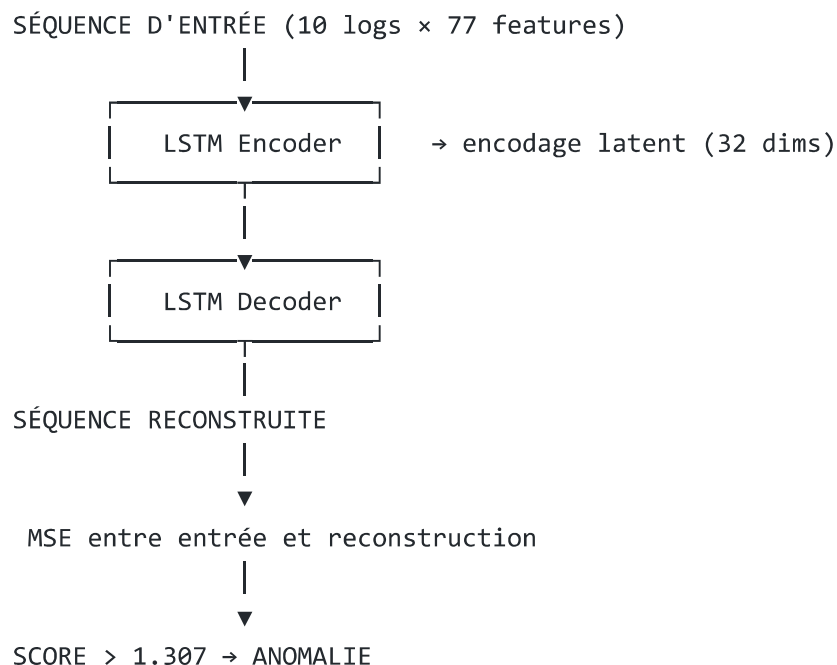
| Paramètre           | Valeur        | Justification           |
|---------------------|---------------|-------------------------|
| BATCH_SIZE          | 500 messages  | Compromis latence/débit |
| BATCH_TIMEOUT_SEC   | 30s           | Garantie de fraîcheur   |
| Compression Parquet | snappy        | Décompression rapide    |
| Group ID Kafka      | etl-processor | Permet scale horizontal |

6. Phase 3 — Modèle de Machine Learning

## 6.1 Choix architectural : LSTM Autoencoder

### Pourquoi un autoencoder ?

Détection d'anomalies en **apprentissage non-supervisé** : on entraîne uniquement sur des séquences "normales", et on identifie les anomalies par leur **erreur de reconstruction** élevée.



## 6.2 Hyperparamètres du modèle

| Paramètre   | Valeur | Description                        |
|-------------|--------|------------------------------------|
| n_features  | 77     | Dimensions du vecteur log          |
| seq_len     | 10     | Taille fenêtre glissante           |
| hidden_size | 64     | Neurones LSTM cachés               |
| latent_size | 32     | Espace latent compressé            |
| vocab_size  | 5000   | Tokens uniques                     |
| embed_dim   | 64     | Embedding dimensions               |
| threshold   | 1.3072 | Seuil percentile 95 sur validation |

## 6.3 Pipeline d'entraînement (offline)

1. **Lecture** : tous les Parquet ETL
2. **Vectorisation** ( `features.py` ) :
  - Embedding du message via vocabulaire 5000 tokens
  - Features numériques : longueur, casse, ponctuation, etc.



- One-hot encoding du level
- Total : **77 dimensions**
- 3. **Scaling** : StandardScaler sklearn (sauvegardé en `feature_scaler.pkl` )
- 4. **Construction de séquences** : sliding window de 10 logs par (source, host)
- 5. **Split** : train (80%) / validation (20%) — uniquement séquences normales
- 6. **Entraînement** : optimizer Adam, MSE loss
- 7. **Calcul du seuil** : percentile 95 sur erreurs de validation
- 8. **Sauvegarde** : 5 artefacts dans S3 `logguardian-models-148761640356`

## 6.4 Inférence en production (online)

```
# Boucle principale ml-model
for message in kafka_consumer:
    sequence = sliding_buffer.add(message) # par (source, host)
    if sequence is None:
        continue # buffer pas encore à 10

    score = model.reconstruction_error(sequence)
    if score > threshold:
        publish_to("logs-anomalies-ml", {
            "anomaly_score": score,
            "severity_ratio": score / threshold,
            "sequence": sequence,
            ...
        })
```

### Sliding Buffer

Implémentation d'un buffer glissant **par couple (source, host)** :

- Permet de traiter chaque host indépendamment
- Évite de mélanger des logs de hosts différents dans une même séquence
- Utilise `collections.deque(maxlen=10)` pour O(1) sur add

### Téléchargement à chaud

Au démarrage du pod, le service télécharge les 5 artefacts depuis S3 :

- `lstm_autoencoder.pt` (poids PyTorch)
- `threshold.json` (métadonnées)
- `vocabulary.pkl` (vocabulaire)
- `embedding_table.npy` (table d'embedding)
- `feature_scaler.pkl` (scaler sklearn)

→ Permet de mettre à jour le modèle sans rebuild l'image Docker.

## 6.5 Performances du modèle

Évaluation sur 645 482 séquences de test (notebook d'évaluation) :

| Classe   | Précision | Rappel | F1-score | Support |
|----------|-----------|--------|----------|---------|
| Normal   | 0.93      | 0.81   | 0.87     | 581 332 |
| Anomalie | 0.21      | 0.46   | 0.29     | 64 150  |
| Accuracy |           |        | 0.77     | 645 482 |

### Matrice de confusion

|        |          | Predicted |          |               |
|--------|----------|-----------|----------|---------------|
|        |          | Normal    | Anomalie |               |
| Actual | Normal   | 470545    | 110787   | (FP: 19%)     |
|        | Anomalie | 34728     | 29422    | (Recall: 46%) |

### Interprétation

- **Précision Normal élevée (93%)** : quand le modèle dit "normal", il a souvent raison
- **Rappel Anomalie modéré (46%)** : on capture environ 1 anomalie sur 2
- **Trade-off** : le seuil au percentile 95 favorise la précision sur les normaux. On pourrait l'abaisser pour augmenter le rappel mais cela générerait plus de faux positifs.

### Loss d'entraînement

- Train loss : **0.2696**
- Val loss : **0.2641**

Le fait que val loss < train loss indique **aucun surapprentissage**. Le modèle généralise bien.

## 7. Phase 4 — Interface de monitoring

### 7.1 Architecture UI

Application **Plotly Dash** (Python) qui :

- Consomme logs-anomalies-m1 via confluent-kafka dans un thread dédié
- Stocke les 2000 dernières anomalies dans un buffer en mémoire
- Affiche un dashboard temps réel avec rafraîchissement toutes les 3 secondes

## 7.2 Composants visuels

### KPI cards (haut du dashboard)

- **Total anomalies** : compteur cumulatif
- **Sévérité haute** : ratio > 1.3x du seuil (rouge)
- **Sources** : nombre de sources distinctes actives
- **Topic** : indication du topic Kafka source

### Tableau "Toutes les anomalies"

Colonnes : Heure, Source, Host, Score, Ratio, Seuil, Dernier log

Tri natif, pagination 50, code couleur sur ratio :

- > 1.3x : rouge (sévérité haute)
- ≤ 1.3x : jaune (sévérité standard)

### Tableau "Sévérité haute" (panneau latéral)

Filtré sur les anomalies critiques uniquement, format compact (Source, Score, Dernier log).

## 7.3 Design

Theme **dark mode** avec palette cohérente :

- Background : #0f1117 (gris très foncé)
- Surface : #1a1d27 (cards/tables)
- Danger : #ef4444 (rouge anomalies)
- Warning : #f59e0b (jaune)
- Text : #e2e8f0 (blanc cassé)

## 7.4 Exposition réseau

Service Kubernetes de type **LoadBalancer** avec annotations spécifiques :

```
service.beta.kubernetes.io/aws-load-balancer-type: "nlb"  
service.beta.kubernetes.io/aws-load-balancer-scheme: "internet-facing"
```

→ AWS provisionne automatiquement un **NLB internet-facing** avec URL publique : `http://k8s-logguard-monitori-xxx.elb.eu-west-1.amazonaws.com`

## 8. Déploiement Kubernetes

## 8.1 Vue d'ensemble des manifests

| Fichier            | Ressources                                   |
|--------------------|----------------------------------------------|
| namespace.yaml     | Namespace logguardian                        |
| kafka.yaml         | Zookeeper + Kafka (Deployment + Service)     |
| log-generator.yaml | SA + ConfigMap + Deployment (init container) |
| etl-processor.yaml | SA + ConfigMap + Deployment                  |
| ml-model.yaml      | SA + ConfigMap + Deployment                  |
| monitoring-ui.yaml | ConfigMap + Deployment + Service LB          |
| kustomization.yaml | Agrégation des manifests                     |

## 8.2 État des pods en production

| Pod           | Status  | Restarts | CPU     | Memory  |
|---------------|---------|----------|---------|---------|
| zookeeper     | Running | 0        | 2m      | 95Mi    |
| kafka         | Running | 0        | 82m     | 690Mi   |
| log-generator | Running | 0        | 121m    | 23Mi    |
| etl-processor | Running | 0        | 115m    | 97Mi    |
| ml-model      | Running | 0        | (varie) | (varie) |
| monitoring-ui | Running | 0        | <100m   | <128Mi  |

## 8.3 Patterns Kubernetes utilisés

### Init Container (log-generator)

```
initContainers:
  - name: sync-log-data
    image: amazon/aws-cli:2.15.57
    command: [sh, -c, 'aws s3 sync "$DATA_S3_URI" /data']
    volumeMounts:
      - name: log-data
        mountPath: /data
volumes:
  - name: log-data
    emptyDir: {}
```

### Strategy Recreate (Kafka)

```
strategy:
  type: Recreate # Évite double-pod conflict avec broker.id unique
```

enableServiceLinks: false

Désactivé pour éviter conflits entre env vars auto-injectées par K8s et config Kafka.

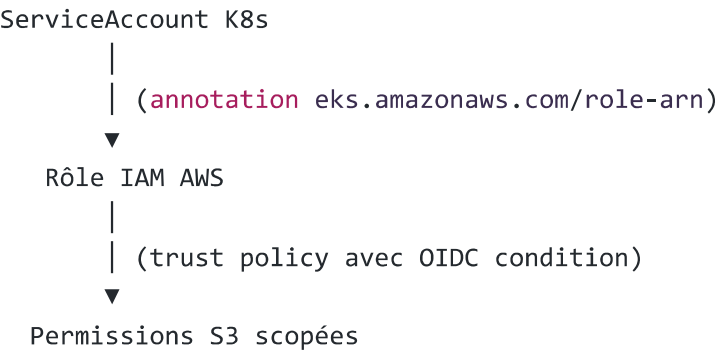
Readiness Probe

```
readinessProbe:
  httpGet:
    path: /
    port: 8050
```

# 9. Sécurité et IAM

## 9.1 Pattern IRSA (IAM Roles for Service Accounts)

Principe : aucun credential AWS hardcodé. Chaque pod assume un rôle IAM spécifique via OIDC.



## 9.2 Rôles IAM créés

| Rôle                        | ServiceAccount            | Permissions                                 |
|-----------------------------|---------------------------|---------------------------------------------|
| LogGuardianLogGeneratorRole | logguardian/log-generator | S3 GetObject + ListBucket sur raw-logs/*    |
| LogGuardianETLProcessorRole | logguardian/etl-processor | S3 full access sur datalake                 |
| LogGuardianMLModelRole      | logguardian/ml-model      | S3 GetObject + ListBucket sur models bucket |

## 9.3 Trust Policy IRSA (exemple)

```
{
  "Effect": "Allow",
  "Principal": {
    "Federated": "arn:aws:iam::148761640356:oidc-provider/oidc.eks.eu-west-1.amazonaws.com/id/"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "...:sub": "system:serviceaccount:logguardian:log-generator",
      "...:aud": "sts.amazonaws.com"
    }
  }
}
```

## 9.4 Sécurité réseau

- Communication inter-services **interne au cluster** (Kafka non exposé extérieurement)
- Seul `monitoring-ui` est exposé via NLB internet-facing
- Pas de secrets dans les manifests : tout via ConfigMaps + IRSA

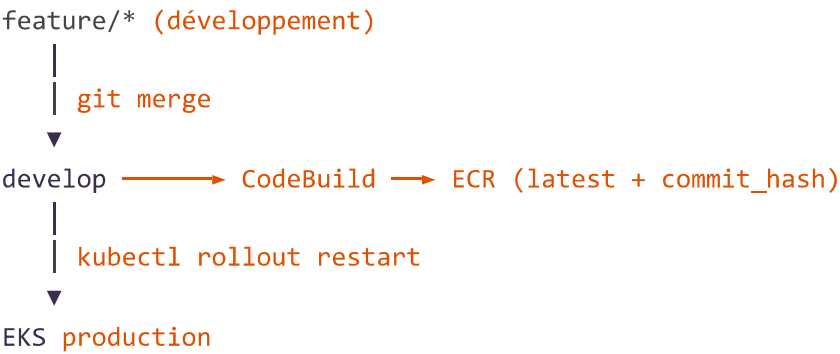
# 10. Pipeline CI/CD

## 10.1 AWS CodeBuild ( `buildspec.yml` )

Déclenché automatiquement sur **merge vers la branche `develop`** :

```
phases:
  pre_build:
    - aws ecr get-login-password | docker login ECR
    - COMMIT_HASH=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c 1-7)
  build:
    - for service in log-generator etl-processor ml-model monitoring-ui:
      if [ -f "$service/Dockerfile" ];
      docker build -t $REPO/$service:$IMAGE_TAG ./service
      docker tag ... :latest
  post_build:
    - docker push $REPO/$service:$IMAGE_TAG
    - docker push $REPO/$service:latest
```

## 10.2 Workflow Git



### 10.3 Stratégie de tagging

- :commit\_hash (ex: 9991ecd ) — traçabilité
- :latest — rolling update simplifié

Les manifests utilisent :latest avec imagePullPolicy: Always pour garantir un pull à chaque restart.

## 11. Résultats et métriques

### 11.1 Performances de production

Snapshot du 01/05/2026 à 19:30 :

| Métrique                 | Valeur             |
|--------------------------|--------------------|
| Logs ingérés totaux      | 1 463 500          |
| Anomalies règles (ETL)   | 105 135 (7.2%)     |
| Anomalies ML             | 14 893             |
| Anomalies sévérité haute | 172 (> 1.3x ratio) |
| Fichiers Parquet S3      | ~3000              |
| Volume datalake          | ~50 MB             |
| Throughput live          | 100 msg/s soutenu  |
| Latence end-to-end       | < 15 secondes      |

### 11.2 Distribution des sources

Selon le segment du cycle de replay actuel :

- **Linux** : ~7-10% d'anomalies (logs majoritairement INFO)

- **SSH** : ~30% d'anomalies (échecs auth fréquents)
- **Hadoop** : ~5% d'anomalies (cluster sain)
- **Supercomputer (BGL)** : 100% d'anomalies (dataset de pannes)

### 11.3 Métriques cluster EKS

| Node            | CPU  | Memory | %         |
|-----------------|------|--------|-----------|
| ip-10-0-137-137 | 162m | 885Mi  | 8% / 26%  |
| ip-10-0-157-96  | 227m | 1378Mi | 11% / 41% |

→ Le cluster a **largement de la marge** pour scaler horizontalement (HPA possible).

## 12. Difficultés rencontrées et solutions

### 12.1 Kafka crash en boucle (CrashLoopBackOff)

**Cause** : variables d'environnement auto-injectées par Kubernetes (services K8s) entraînent en conflit avec config Kafka.

**Solution** : `enableServiceLinks: false` dans le pod spec.

### 12.2 Double pod Kafka pendant rolling update

**Cause** : `strategy: RollingUpdate` (par défaut) crée le nouveau pod avant de supprimer l'ancien → conflit `broker.id`.

**Solution** : `strategy: Recreate` pour Kafka (1 broker à la fois).

### 12.3 Container `etl-processor` `NoCredentialsError` sur S3

**Cause** : ServiceAccount sans annotation `IRSA` → aucune identité AWS.

**Solution** : création du rôle IAM `LogGuardianETLProcessorRole` avec trust policy `OIDC` + annotation sur le SA.

### 12.4 Init container échec avec `InvalidIdentityToken`

**Cause** : `OIDC` provider du cluster pas enregistré dans IAM (étape manquante).

**Solution** : ajout via console IAM → Identity providers → OpenID Connect.

### 12.5 LoadBalancer <pending> sur EKS Auto Mode



**Cause** : EKS Auto Mode utilise `EKSNetworkingChainRole` (compte AWS managé) qui devait pouvoir assumer notre rôle cluster avec `sts:TagSession` .

**Solution** : ajout d'une statement dans le trust policy de `logguardian-eks-cluster-role` .

## 12.6 NLB créé en mode interne (DNS non résolvable)

**Cause** : annotation NLB par défaut crée un load balancer privé.

**Solution** : ajout `service.beta.kubernetes.io/aws-load-balancer-scheme: "internet-facing"` .

## 12.7 Log-generator perd sa progression à chaque restart

**Cause** : pas de mécanisme de checkpoint, replay reprenait depuis le début.

**Solution** : implémentation d'un système de checkpoint S3 :

- Sauvegarde toutes les 1000 messages
- Sauvegarde au SIGTERM (graceful shutdown)
- Skip rapide au redémarrage (sans sleep)

# 13. Améliorations futures

---

## 13.1 Court terme (Phase 5)

1. **Migration entraînement vers SageMaker** : entraînement managé, scaling auto
2. **AutoScaling HPA** : scaling horizontal basé sur lag Kafka consumer
3. **Métriques Prometheus + Grafana** : observabilité technique avancée
4. **Alerting** : Slack/PagerDuty webhook depuis monitoring-ui

## 13.2 Moyen terme

1. **Multi-modèles par source** : un LSTM dédié par type de log (Linux  $\neq$  SSH  $\neq$  Hadoop)
2. **Online learning** : mise à jour incrémentale du modèle sans redéploiement
3. **Explainability** : indiquer **quelle partie** de la séquence est anormale (attention weights)
4. **Intégration LLM** : enrichir la description des anomalies via Claude/GPT

## 13.3 Production-grade

1. **Multi-AZ** : déploiement sur 3 zones de disponibilité
2. **Backup S3 cross-region** : disaster recovery
3. **Authentication UI** : OAuth/Cognito pour le dashboard

4. TLS partout : ALB Ingress avec certificats ACM

# 14. Conclusion

## 14.1 Bilan technique

Le projet LogGuardian démontre la mise en œuvre d'une **plateforme AIOps complète** combinant :

- **Architecture microservices event-driven** (Kafka)
- **Cloud-native** (Kubernetes, S3, ECR, IAM IRSA)
- **CI/CD automatisé** (CodeBuild)
- **Deep Learning** (LSTM Autoencoder PyTorch)
- **Visualisation temps réel** (Plotly Dash)

L'ensemble est en **production live** sur AWS EKS avec **plus de 1.4 million de logs traités**.

## 14.2 Apports pédagogiques

Ce projet a permis de manipuler concrètement :

| Domaine       | Compétences acquises                           |
|---------------|------------------------------------------------|
| Cloud AWS     | EKS, S3, ECR, IAM, CodeBuild, NLB              |
| Conteneurs    | Docker, Kubernetes, init containers, IRSA      |
| Streaming     | Kafka producer/consumer, micro-batching        |
| Big Data      | Parquet, partitioning, S3 datalake             |
| Deep Learning | PyTorch, LSTM, autoencoders, MSE loss          |
| MLOps         | Model artifacts, S3 model registry, hot-reload |
| DevOps        | Git workflow, CI/CD, infrastructure as code    |
| Sécurité      | IAM least-privilege, OIDC federation           |
| Networking    | Service Kubernetes, LoadBalancer AWS           |

## 14.3 Originalité et valeur ajoutée

Contrairement à des solutions de marché type **Splunk** ou **Datadog** (basées sur règles + recherche full-text), LogGuardian utilise un **modèle séquentiel** capable de détecter des anomalies **contextuelles** :

Un log isolé n'est pas anormal, mais sa présence après une certaine séquence l'est.

C'est précisément ce que le LSTM Autoencoder capture, en apprenant les patterns normaux de séquences de logs par host.

## 15. Annexes

### Annexe A — Structure du repository

```
logguardian/
├── README.md
├── buildspec.yml                # CI/CD CodeBuild
├── docker-compose.yml          # Stack dev locale
├── LogGuardian_Phase1_Infrastructure.md
├── LogGuardian_Phase2_Pipeline.md
├── RAPPORT_PROJET_LOGGUARDIAN.md # Ce document
├── data/                        # Datasets sources (LogHub)
│   ├── Linux.log
│   ├── SSH.log
│   ├── hadoop/
│   └── supercomputer/BGL.log
├── k8s/                         # Manifests Kubernetes
│   ├── namespace.yaml
│   ├── kafka.yaml
│   ├── log-generator.yaml
│   ├── etl-processor.yaml
│   ├── ml-model.yaml
│   ├── monitoring-ui.yaml
│   └── kustomization.yaml
├── log-generator/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── README.md
│   ├── src/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── checkpoint.py
│   │   ├── producer.py
│   │   └── parsers/
│   │       ├── base.py
│   │       ├── linux_parser.py
│   │       ├── ssh_parser.py
│   │       ├── hadoop_parser.py
│   │       └── supercomputer_parser.py
│   └── tests/
├── etl-processor/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── README.md
```

```

├── src/
│   ├── main.py
│   ├── config.py
│   ├── consumer.py
│   ├── producer.py
│   ├── transformer.py
│   └── s3_loader.py
├── ml-model/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── README.md
│   ├── notebook_evaluation.ipynb
│   ├── models/                                # Artefacts (uploadés sur S3)
│   │   ├── lstm_autoencoder.pt
│   │   ├── threshold.json
│   │   ├── vocabulary.pkl
│   │   ├── embedding_table.npy
│   │   └── feature_scaler.pkl
│   └── src/
│       ├── main.py
│       ├── config.py
│       ├── consumer.py
│       ├── producer.py
│       ├── inference/
│       │   ├── buffer.py
│       │   └── detector.py
│       └── trainer/
│           ├── model.py
│           ├── features.py
│           ├── dataset.py
│           └── train.py
├── monitoring-ui/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── README.md
│   └── src/
│       └── app.py

```

## Annexe B — Commandes utiles d'exploitation

```

# Démarrage du cluster (dans l'ordre)
kubectl scale deployment zookeeper -n logguardian --replicas=1
# attendre 15s
kubectl scale deployment kafka -n logguardian --replicas=1
# attendre 30s
kubectl scale deployment log-generator etl-processor ml-model monitoring-ui -n logguardian --r

# Arrêt complet
kubectl scale deployment monitoring-ui ml-model log-generator etl-processor kafka zookeeper -n

# Surveillance

```

```
kubectl get pods -n logguardian
kubectl top pods -n logguardian
kubectl logs -n logguardian -l app=ml-model -f --tail=50

# Vérifier datalake
aws s3 ls s3://logguardian-datalake-148761640356/logs/ --recursive --summarize | Select-Object

# URL dashboard
kubectl get svc monitoring-ui -n logguardian
```

## Annexe C — Variables d'environnement principales

### log-generator

```
KAFKA_BOOTSTRAP_SERVERS=kafka.logguardian.svc.cluster.local:29092
KAFKA_TOPIC=logs-raw
DATA_DIR=/data
DATA_S3_URI=s3://logguardian-datalake-148761640356/raw-logs/
LOG_SOURCES=linux,ssh,hadoop,supercomputer
S3_BUCKET=logguardian-datalake-148761640356
AWS_REGION=eu-west-1
REPLAY_SPEED=100
```

### etl-processor

```
KAFKA_INPUT_TOPIC=logs-raw
KAFKA_OUTPUT_TOPIC=logs-processed
KAFKA_ANOMALY_TOPIC=logs-anomalies
S3_BUCKET=logguardian-datalake-148761640356
S3_PREFIX=logs
BATCH_SIZE=500
BATCH_TIMEOUT_SEC=30
```

### ml-model

```
KAFKA_INPUT_TOPIC=logs-processed
KAFKA_OUTPUT_TOPIC=logs-anomalies-ml
S3_MODELS_BUCKET=logguardian-models-148761640356
LOCAL_MODEL_MODE=false
MODEL_DIR=/app/models
SEQUENCE_LENGTH=10
DEVICE=cpu
```

### monitoring-ui

```
KAFKA_BOOTSTRAP_SERVERS=kafka.logguardian.svc.cluster.local:29092  
KAFKA_TOPIC=logs-anomalies-ml  
MAX_ROWS=2000  
REFRESH_INTERVAL_MS=3000
```

## Annexe D — Captures d'écran (à inclure)

1. Dashboard monitoring-ui en production
2. Console AWS EKS — pods Running
3. Console S3 — datalake Parquet partitionné
4. Console ECR — 4 images poussées
5. Console CodeBuild — historique de builds
6. Notebook évaluation modèle — matrice de confusion

## Fin du rapport

*LogGuardian — Plateforme AIOps de détection d'anomalies ESGI — Projet Annuel 2025/2026*